AVX2-Accelerated Brightness Comparison for Choosing a Video Compression Strength

Juan Miguel Miranda; Jean Khladyn Stefanie Abarquez; and Kimberly Klaire Gamboa

0. Abstract

This project aims to evaluate how data-level parallelism using AVX2 assembly can accelerate brightness comparisons between consecutive video frames. A screen recording is often easy to compress, except for a short flash of dense text, such as a slide, a menu, or a paragraph. That moment may last less than a second. A check that looks at only a few frames spread through the clip can miss it. The encoder may then compress the whole file more harshly, and that text is the part most likely to lose quality first.

This project will look for the moment the picture changes by comparing the characteristics of each frame with the next, such as the brightness of the pixels. The comparison will be evaluated using three implementations: ordinary sequential C, compiler-optimized C, and AVX2 assembly. These implementations will be compared to determine how much execution time is reduced through compiler optimization and how much additional improvement can be achieved through explicit SIMD parallelism. All three implementations must produce identical frame-comparison results and detect the same scene changes.

After a change is detected, the check waits briefly so the frames it looks at are the new picture. It then chooses the strongest compression those frames still allow while meeting a quality target, which will then be used for the whole file. The project aims to determine how much faster AVX2 assembly can perform the brightness comparisons compared with ordinary and compiler-optimized C, while producing the same results. It will also examine whether using the detected changes to select frames can help preserve the quality of brief, dense-text content during compression.

1. Description

The clip is thirty seconds long. It shows a simple still picture, such as a plain slide, cuts to a page of dense text for twenty frames, and cuts back. There is no fade. The program is not told when the text appears. It only sees the frames.

The way of choosing how to compress the file could be through a regular interval or through the analysis of the next image detected. Each way picks a compression strength that still meets a quality target. The finished file is then judged by a score of the whole clip to analyze the quality and size of the file.

For the integrating project, the primary focus will be on measuring the speed of detecting these changes through brightness comparisons. The compression process will demonstrate how the detected changes can be used to select frames for determining the compression strength.

a. Inputs

- A screen clip, 1920 × 1080 pixels at 30 frames per second, for a total of 900 frames.
- An easy picture and a dense-text picture for that clip. The text is held for 20 frames.
- The characteristics of each frame, particularly pixel brightness values, so neighboring frames can be compared.

- A quality target. Compression is done with different strengths. A rule may pick a strength only when it meets the target.
- The same decoded frame data will be provided to all three implementations to ensure a fair comparison of their execution times and results.

b. Process

i. What processes will be parallelized using SIMD-based AVX2 assembly?

The primary process to be parallelized is the computation of brightness differences between corresponding pixels of consecutive video frames. For each pixel, the brightness values from two neighboring frames are compared by calculating their absolute difference. These differences are combined into a single result representing how much the frame has changed. Since a 1920 × 1080 frame contains more than two million pixels, the computation requires a large number of repeated operations.

The sequential C implementation will compare pixel brightness values one at a time, while the AVX2 assembly implementation will aim to process multiple pixel values simultaneously. By allowing several comparisons to be performed at once, the AVX2 implementation is expected to reduce the execution time of detecting changes between frames while maintaining the same results as the C implementation.

The program will compare each frame with the one immediately before it by checking the differences in their pixel brightness values. Since each pair of pixels can be compared separately, multiple comparisons can be performed at the same time. These differences will then be combined into a single result to determine how much the picture has changed and whether a scene change has occurred. This process will continue for each pair of consecutive frames throughout the video.

ii. What existing implementations have been done with the proposed process? How would it differ from the proposed implementation?

A scorer for compressed video already exists. It compares each compressed frame with the original and gives that frame one score. A higher score means the frame is closer to the original. Based on the work of Yang et al. (2025), a short bad moment can matter even when most of the clip looks fine. Not every frame is equally important, and a short dense page can hold the part a viewer needs. An average over the whole clip can hide the damage to that segment. This project will keep a score of the finished file that can still see that short page. A separate check, looking at a few frames spread through the clip, will be used for comparison.

There is existing work on choosing how hard to compress a video while still meeting a quality target. Based on the work of Mico-Enguidanos et al. (2023), a video can be split into short pieces, and each piece can be given its own strength so that the piece still meets a chosen quality. Li, Yu, and Wang (2022) save space by changing the compressor itself so that sharp text and sudden changes of pages are treated more carefully than the rest. For the scope of this research, the compressor will not be changed, and one compression strength will be assigned to the whole file, chosen based on the frames observed after a scene change.

There is already a standard way to find a sudden change by comparing one frame with the next. Based on the work of Chen et al. (2002), that comparison is the first step. If the new picture lasts only a few frames, they count it as still belonging to the picture before it. Based on the work of Smeaton, Over, and Doherty (2009), a hard jump can be found reliably this way, while a slow fade is harder. This project will

use a simple brightness comparison and will not claim a new way to find cuts. The planned clip has no fade, and the text has to stay on screen long enough for the check to see it.

For the computer architecture component, the project will compare three implementations of the same brightness-comparison algorithm: ordinary C, compiler-optimized C, and AVX2 assembly. Compiler optimizations can already reduce the execution time of a sequentially written program, including by automatically generating vector instructions when possible. Therefore, comparing AVX2 assembly only against unoptimized C would not fully demonstrate the additional advantage of manually implementing SIMD operations.

The proposed implementation will distinguish the performance improvement obtained through compiler optimization from the additional improvement achieved by explicit AVX2 assembly. The AVX2 implementation is expected to execute faster than both C versions while returning identical pixel-difference results and detected scene changes.

The difference in the compression application is in which frames are allowed to choose the compression strength. This project will wait until a sudden change has settled on the new picture, and those frames will choose one strength for the whole file. A few frames spread through the clip serve as the comparison because that spread can miss a short page of dense text and compress the whole file as if the page were as easy as the rest. The scorer, the compressor, and the method of finding cuts will use existing approaches.

iii. Discuss plans on how to parallelize the proposed process

The brightness comparison will be implemented and evaluated in three versions:

1. **Ordinary C implementation.** This version uses a conventional loop that calculates the brightness difference between corresponding pixels. It will serve as the sequential baseline, with automatic compiler vectorization disabled for the benchmark.
2. **Compiler-optimized C implementation.** This version uses the same C algorithm but enables compiler optimizations, allowing GCC to improve the execution of the program. Its performance will show how much execution time can already be reduced without manually writing AVX2 assembly.
3. **AVX2 assembly implementation.** This version explicitly uses SIMD instructions to process multiple pixel brightness values simultaneously. Vector operations will calculate and accumulate differences, with any remaining pixels handled separately when necessary.

All three implementations will receive identical frame data and must return the same total pixel-difference values and scene-change positions. This ensures that performance improvements do not come at the cost of changing the output.

Execution time will be measured using the following benchmarking procedure:

- The time comparison will cover only the frame brightness-comparison computation. Video decoding, encoding, and quality scoring will be excluded from the timing measurements since these processes are not part of the operation being parallelized.
- Each implementation will be tested multiple times using the same input data to account for variations in execution time caused by hardware variances.

- The reported execution time will be the median, or middle value, of the repeated measurements rather than the result of a single run.
- Speedup will be calculated by comparing the execution times of the ordinary C, compiler-optimized C, and AVX2 assembly implementations.

The expected result is that compiler-optimized C will substantially reduce execution time relative to ordinary C, while the AVX2 assembly implementation will achieve additional improvement over the compiler-optimized version. The significance of these improvements will be determined through benchmarking.

iv. What other processes are in the project but will not be parallelized?

The following processes will remain sequential and outside the AVX2 computation:

1. The moment the text appears will be chosen and written down.
2. The video will be decoded into frames for analysis.
3. The three brightness-comparison implementations will be executed and benchmarked using the same frame data.
4. The detected scene changes will be used to identify frames for compression analysis.
5. The video will be encoded using different compression strengths.
6. The compressed versions will be scored to determine their quality.
7. The compression strength will be selected based on the scores obtained from the sampled frames.

The AVX2 optimization will apply specifically to the repeated pixel-level brightness comparisons. Frame decoding, compression, quality evaluation, and the final selection of compression strength will not be included in the parallel execution-time comparison.

c. Outputs

The project will produce the following:

- Pixel-difference results and scene-change positions from ordinary C, compiler-optimized C, and AVX2 assembly.
- An execution-time comparison table showing the median runtime of all three implementations.
- Calculated speedups demonstrating the improvements from compiler optimization and AVX2 parallelism.
- The compression strength chosen for the video, together with the resulting quality scores and file sizes.
- Final source code and a discussion of the implemented parallel algorithm and its performance.

2. Proposed Target Implementation Platform

The implementation and timed runs will be performed using the following platform:

Component	Specification
Processor	AMD Ryzen 9 5900HS
Memory	16 GB RAM

Operating System	Windows 10, 64-bit
Compiler	GCC
SIMD Instruction Set	AVX2
Programming Languages	C and x86-64 assembly

The same hardware platform and input data will be used for all three implementations to maintain consistency in the execution-time comparisons.

3. Mid-Project Milestones of the Proposed Project

1. A functional system that accepts screen recordings, detects scene changes, selects representative frames, and uses those frames to determine an appropriate compression strength.
2. Working implementations of the brightness-comparison process using ordinary C, compiler-optimized C, and AVX2 assembly.
3. Verification that all three implementations produce identical pixel-difference results and detect the same scene changes.
4. A preliminary execution-time comparison showing the median runtime of each implementation and the speedups achieved through compiler optimization and AVX2 parallelism.
5. Testing the system on several screen recordings with different visual characteristics to evaluate whether it can consistently detect changes and complete the compression process.
6. A comparison of the resulting video quality and file sizes when compression strength is selected using regularly spaced frames versus frames selected after detected scene changes.

By this milestone, the main processing pipeline is expected to be operational, with preliminary results available for both execution speed and video compression quality. Further testing and evaluation will be conducted to improve the reliability of the results and support the final project submission.

4. References

1. S. Yang, J. Ji, Q. Zhu, Z. Yang, and X. Jin, “No-reference rendered video quality assessment: dataset and metrics,” arXiv:2510.13349, 2025.
2. F. Micó-Enguídanos, W. Moina-Rivera, J. Gutiérrez-Aguado, and M. Garcia-Pineda, “Per-title and per-segment CRF estimation using DNNs for quality-based video coding,” *Expert Systems with Applications*, vol. 227, article 120289, 2023.
3. J. Li, L. Yu, and H. Wang, “Perceptual redundancy model for compression of screen content videos,” *IET Image Processing*, vol. 16, pp. 1724–1741, 2022.
4. S.-C. Chen, M.-L. Shyu, W. Liao, and C. Zhang, “Scene change detection by audio and video clues,” *IEEE International Conference on Multimedia and Expo*, 2002.
5. A. F. Smeaton, P. Over, and A. R. Doherty, “Video shot boundary detection: seven years of TRECVid activity,” preprint, 2009.
6. N. Safonov, M. Rakhmanov, D. Vatolin, R. Timofte, and participating teams, “AIM 2025 challenge on screen-content video quality assessment: methods and results,” *ICCV Workshops*, 2025.